Bundled for Success: A WfBench-Driven Assessment of Texera

Meng Wang
University of California, Irvine
Irvine, USA
mengw15@uci.edu

Shiva Jahangiri
Santa Clara University
Santa Clara, USA
sjahangiri@scu.edu

Ketan Maheshwari
Oak Ridge National Lab
Oak Ridge, USA
maheshwarikc@ornl.gov

Chen Li
University of California, Irvine
Irvine, USA
chenli@ics.uci.edu

Abstract

Modern workflow systems need rapid evolution in the wake of unprecedented growth and changes in the computing landscape due to the proliferation of seamless AI/LLM usage and the need for distributed collaborative environments. We present Texera, a performant multi-modal workflow management system that can schedule and run complex workflows efficiently on a variety of systems. We integrate WfBench, a popular workflow benchmarking system with Texera, to demonstrate the system's performance, capabilities, and features. In the process, we describe our approach to successfully integrating such systems and the lessons learned. We believe these efforts are informative to the workflows community undertaking similar integrations with benchmark suites.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; **Parallel computing methodologies**; **Distributed computing methodologies**.

Keywords

Workflow Systems, Benchmarking, HPC

ACM Reference Format:
Meng Wang, Shiva Jahangiri, Ketan Maheshwari, and Chen Li. 2025. Bundled for Success: A WfBench-Driven Assessment of Texera. In *54th International Conference on Parallel Processing Companion (ICPP Companion '25), September 08–11, 2025, San Diego, CA, USA.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3750720.3757297

1 Introduction

Composed science workflows offer an end-to-end solution to complex problems. They form an automated executable and reproducible unit of communication for a science campaign. Scientific workflow management systems (WMS) interpret, orchestrate, and organize such science campaigns in the form of workflows. These campaigns have become increasingly complex due to the dynamic nature of user collaborative environments. Additionally, modern computing involves methods that may be a mix of local and remote cloud-based service calls. In particular, for the campaigns that require AI/LLM functionalities, it has become necessary to run them remotely over a provider's server, such as Google's Gemma or Microsoft's Pilot. Additionally, the availability of AI/LLM capabilities has added to the complexities of the workflows as well due to their special accessibility and interactive usage modes. Our work is motivated by the changing landscape in modern scientific computing brought by AI/LLM being seamlessly utilized, which calls for a need for a "social" workflow management paradigm. These requirements call for a WMS to be ready to integrate a wide variety of functionality and make it available to its users in a frictionless way. At the same time, the WMS needs to be performant and responsive in performing its core functionality, that is, efficiently orchestrating a scientific campaign.

Such performant workflow systems are likely to play a key role in enabling the "laboratories of the future" where users' requirements are specified via a GUI or a text-based representation and the WMS executes the workflow while keeping the user updated on progress and invoking the appropriate remote and local services. As more and more WMS are being developed and released, it stands to the best judgment of the users as to which one to select among the offerings available. One of the key criteria that users often use to judge any system, let alone a workflow system, is by looking at its performance. Thankfully, the workflow community has developed a comprehensive suite of workflow benchmarks that, when used with a given WMS, help users make a judgment as to whether a given workflow system is suitable for their science.

To this end, we present a WMS called "Texera" and its integration with WfBench [6] and demonstrate the benchmark performance on a couple of systems. Integrating and interoperating multiple software systems has been a long-standing challenge across all domains. We present our approach, design decisions, and lessons learned in integrating a workflow system with a prominent workflow benchmarking framework that may be informative to other similar future endeavors. The contributions of this work are as follows:

(1) Integration of the Texera WMS and WfBench describing the design and development process;
(2) Enactment of two representative workflows via the WfBench, namely the Blast and Epigenomics workflows; and
(3) Execution and presentation of detailed performance results over the two distinct systems.

The rest of the paper is organized as follows. Section 2 presents the related work. Section 3 introduces the Texera WMS. Section 4 presents the integration between Texera and WfBench. Section 5 presents experimental results on two distinct architectures and operating systems. Finally, Section 6 presents conclusions and future work.

ICPP Companion '25, San Diego, CA, USA

ACM ISBN 979-8-4007-2109-0/25/09
https://doi.org/10.1145/3750720.3757297

2 Related Work

There are many (possibly more than 300) workflow management systems (WMSs) today, each designed with different goals, interfaces, and target user communities [2]. GUI-based platforms such as Galaxy [8] and Taverna [26] enable domain scientists to design workflows visually, enhancing accessibility but sometimes at the expense of scalability or flexibility. Script-driven systems such as Swift/T [27], Pegasus [9], Parsl [4], and Nextflow [22] support reproducible and distributed data processing across HPC and cloud environments, targeting technically proficient users. Systems such as Apache Airflow [3] and Dask [20] combine programmatic APIs with visual monitoring tools, appealing to data engineers managing production pipelines. Texera [24] offers a hybrid model that supports both batch and streaming workflows, collaborative development, and extensible operators through a web-based interface, making it suitable for cloud deployments.

Given the diversity of these systems, benchmarking and evaluation play a critical role in understanding trade-offs across performance, usability, and reproducibility. Leipzig conducted a survey of bioinformatics pipeline frameworks [17], while Jackson et al. compared the performance and provenance features of Pegasus, Swift/T, and Kepler [14]. Other studies include Diercks et al.'s assessment of workflow description and reuse tools [10]. WfCommons [7] provides tools for modeling realistic synthetic workflows based on real execution traces. Building on that, WfBench [6] supports automatic generation of parameterizable workflow benchmarks and translation into systems such as Swift/T, Pegasus, Dask, and TaskVine. Demonstrating its scalability, WfBench has been used to execute benchmarks with up to 100,000 tasks on an OLCF Summit supercomputer, enabling reproducible and cross-platform workflow evaluation at scale. Additionally, the workflow community [13] regularly organizes summits and workshops to discuss challenges, solutions, and advances in WMS, providing valuable insights and resources for benchmarking and evaluating workflow systems.

3 Texera Workflow Management System

Texera [21] is an open-source workflow system we developed since 2016. It supports collaborative data science using workflows with a drag-and-drop GUI interface, as well as a client-side library for developers to submit workflows using its Python API. Users with various backgrounds, including those with or without coding skills, can collaborate and participate in the entire life cycle of data science. Texera has rich features that allow advanced programmers to add operators in languages (Python, R, Java, and Scala) and debug the execution of a workflow. Figure 1 shows a screenshot of Texera's user interface in which two users collaboratively construct and execute a data science project as a workflow [19]. As of July 2025, it serves more than 600 users working on 100+ projects, with over 3,000 workflows, 51,000 executions, and 273,000 versions. It has multiple deployments, including one on a cluster of 100 computing nodes with 400 cores.

In the past few years, utilizing the Texera system, we successfully led more than multiple joint projects with researchers from various scientific fields, including public health, neuroscience, biology, AI/ML, bioinformatics, and social science. These projects addressed diverse topics such as social media analysis, mask-wearing, COVID-19 vaccination, tobacco use, climate change, wildfires, and incarceration [11, 12, 15]. Researchers from multiple disciplines used the Texera system to share datasets and workflows, enabling them to collaboratively construct and modify different components of the same workflow using their domain expertise. The robust features of Texera significantly enhanced collaboration and productivity. In addition to supporting AI/ML-based data science, Texera also uses state-of-the-art AI/ML techniques to improve its usability and performance. For example, it has a chat bot powered by a large language model (LLM) to allow users to formulate a workflow using natural language text. We are also in the process of using reinforcement learning to decide the optimal parallelism of operators to minimize the execution time. More AI/ML techniques are being developed in the system.

4 WfBench Integration

4.1 Motivation

Our goal is to introduce Texera to the workflow and HPC community by demonstrating its capabilities through standardized benchmarking methodologies. Workflow systems often lack uniform, reproducible benchmarks, making objective comparisons challenging. To solve this problem, WfBench provides structured frameworks and diverse workflow instances.

Integrating WfBench with Texera has several benefits for introducing the system to the broad community. First, WfBench enables comparative evaluation by allowing Texera workflows to be executed and analyzed alongside those from other workflow systems using identical workflow definitions. WfBench offers workflow definitions tailored to various domains, such as bioinformatics and astronomy, providing an opportunity for Texera to demonstrate its strengths in diverse specialized areas. Second, WfBench has a comprehensive performance analysis. WfBench's ability to generate workflows comprising hundreds or thousands of tasks allows for rigorous testing of Texera's scalability, providing insights into the system's performance under realistic and demanding workloads. This helps to demonstrate Texera's resource management and parallel execution efficiency. Additionally, the parameterization capabilities provided by WfBench allow profiling of CPU usage, memory consumption, and I/O operations for individual workflow tasks. Such detailed analysis showcases Texera's ability to remain robust and efficient across diverse computational scenarios.

4.2 Design Principles and Decisions

4.2.1 Design Principles. The integration process leverages several core components provided by the WfCommons framework [7], which includes WfBench. Figure 2 illustrates the interactions among these core components.

Principle 1: Utilizing standardized workflow collections. We utilize the WfInstances component [5], which maintains a curated collection of open-access workflow instances from various scientific applications. Each instance, documented in a standardized format known as "WfFormat," originates from real execution logs collected on distributed computing platforms such as clouds, grids, and clusters, providing transparency and reproducibility for workflow evaluations.

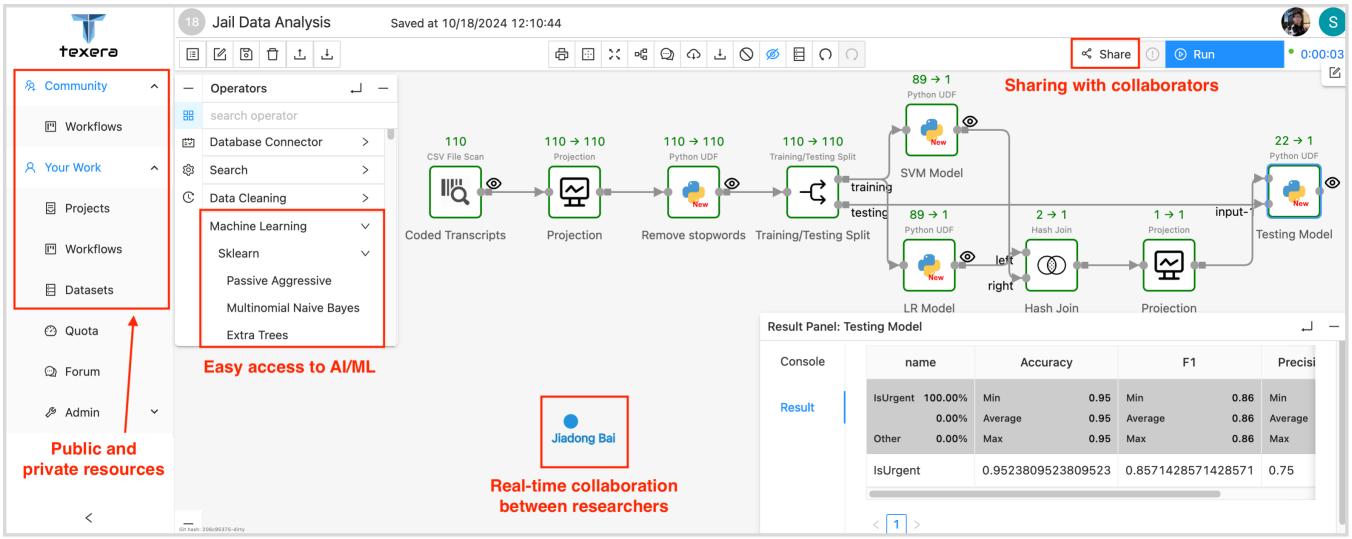

Figure 1: A Texera interface showing two users collaborating on a data science project.

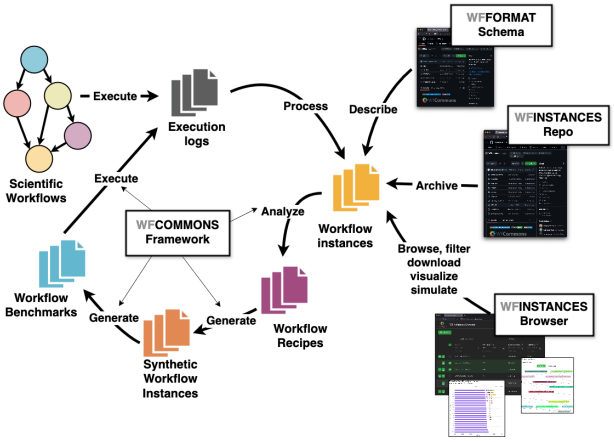

Figure 2: Overview of WfCommons framework components (image adopted from WfCommons website [25]).

Principle 2: Generating realistic synthetic workflows. The WfChef component automates the generation of synthetic workflow generators or "recipes" by analyzing real-world workflow instances provided by WfInstances. Building upon these recipes, WfGen generates realistic, synthetic workflow instances at a user-defined scale. By maintaining fidelity to the structural and performance characteristics observed in real-world scenarios, WfGen enables systematic performance testing and scalability analysis.

Principle 3: Translating workflow recipes into executable workflows. The WfBench component translates workflow recipes into detailed, executable benchmark specifications for existing workflow management systems [5]. It precisely defines tasks' CPU, memory, and I/O usage. Currently, WfBench translators include various workflow systems such as Pegasus, Swift/T, TaskVine, and Dask. Our research extends this capability by introducing a Texera-specific Translator. The Texera-specific Translator transforms workflow recipes into Texera-compatible JSON structures, systematically mapping tasks to Texera operators and explicitly defining inter-operator dependencies.

4.2.2 Decision on Workflow Selection. To demonstrate the effectiveness of our integration approach, we selected two workflows, BLAST and Epigenomics, for the experiments due to their distinctive structural characteristics and domain-specific relevance. These workflows collectively represent common, yet contrasting, workflow structures encountered in practical use cases. Due to space limitations, we focus on these two representative workflows.

The BLAST workflow, modeled after the well-known Basic Local Alignment Search Tool (BLAST) [1], exhibits a clear fan-out and fan-in directed acyclic graph (DAG) structure comprising three distinct stages: splitting input data, parallel computation, and result aggregation. In real-world scenarios, this workflow closely resembles various parallel bioinformatics pipelines and parallel search applications. The structured parallelism in this workflow makes it highly suitable for benchmarking parallel execution and assessing system overhead in managing concurrent tasks.

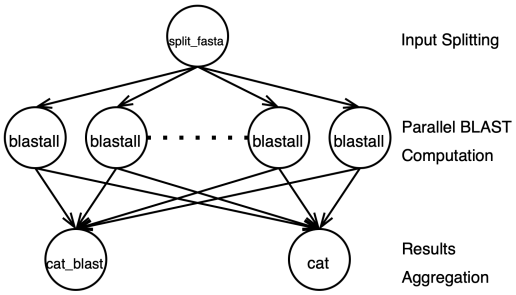

Figure 3: Structure of the BLAST workflow

Figure 3 illustrates the structural organization of the BLAST workflow. Initially, the workflow divides a large FASTA file into multiple smaller input files through the *split_fasta* task. Next, multiple parallel instances of the *blastall* task independently execute computationally intensive sequence alignment operations on these segmented input files. Finally, the workflow aggregates the alignment results via the *cat_blast* task.

In contrast to the fan-out and fan-in structure of the BLAST workflow, the Epigenomics workflow, modeled after realistic bioinformatics pipelines commonly used in genomic and epigenetic data analyses, is structured as a DAG, consisting of multiple sequential processing stages. This workflow represents those with computational characteristics of large-scale parallel and sequential tasks and data-processing demands. These features make this workflow particularly suitable for benchmarking Texera's capabilities in terms of scheduling efficiency, task parallelism, data handling throughput, and computational scalability.

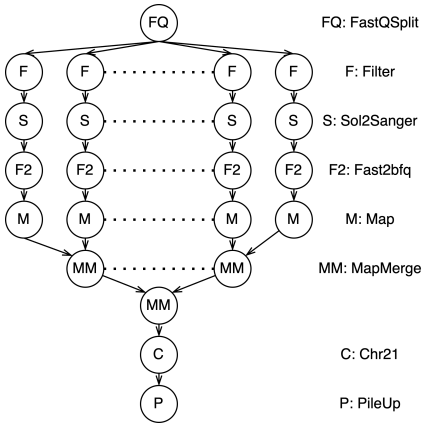

Figure 4: Structure of the Epigenomics workflow

Figure 4 illustrates the detailed structure of the Epigenomics workflow. Initially, large FASTQ files are partitioned into multiple segments through the *FastQSplit* task. Each segment then undergoes parallel preprocessing operations, including contaminant filtering (*Filter*) and sequence format conversion using the *Sol2Sanger* and *Fast2bfq* tasks to optimize computational efficiency for downstream analyses. Subsequently, the workflow performs parallel mapping and alignment via the *Map* tasks. Afterward, alignment outputs are merged through the *MapMerge* tasks, specific genomic regions are extracted by the *Chr21* task, and the processed data is then consolidated into a summary by the *PileUp* task. We evaluate both workflows with three task sizes and three CPU/data configurations. See Section 5 for details.

4.3 Translator Design and Challenges

A key challenge during the integration was designing an appropriate translator to convert workflows generated by WfBench into Texera-compatible workflows. WfBench produces workflows in a generic workflow benchmark object format, necessitating platform-specific translators. Existing translators supported platforms such as Swift/T, TaskVine, Dask, and Pegasus. To integrate Texera into this ecosystem, we needed to develop a translator for it.

We explored two primary design strategies for the translator. The first one involves encapsulating tasks from the same workflow level within a single Texera operator. Figure 5 illustrates this approach, where tasks at the same level are grouped into a single operator. Specifically, Operator B1 encapsulates the single task *split_fasta*, Operator B2 groups multiple parallel *blastall* tasks, and Operator B3 encapsulates the tasks of *cat_blast* and *cat*. This strategy simplifies workflow management by reducing the overall complexity

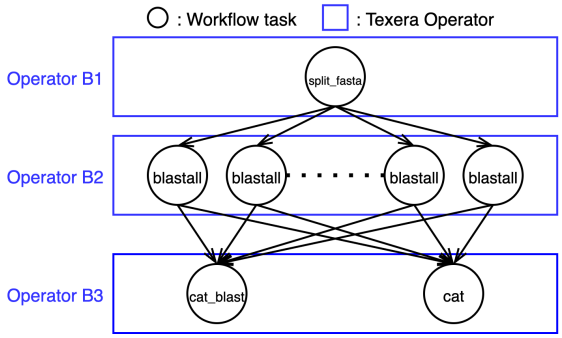

Figure 5: *Encapsulating each workflow level as a Texera operator*

associated with many operators and eases data handling between operators, as there are fewer points of inter-operator data transmission compared to strategies that use individual operators per task. However, internal data transmissions among tasks are still present. Fewer points of inter-operator data transmission can itself be a limitation as it reduces flexibility in managing individual tasks independently. Moreover, this approach has some other drawbacks, primarily semantic misalignment with Texera's design principle, which expects each operator to perform a well-defined and consistent operation across all its workers. Since tasks grouped at the same level may perform diverse operations, parallel execution within a single operator conflicts with Texera's design principle. Furthermore, this strategy reduces visibility into individual workflow tasks, complicating the use of Texera's interactive capabilities, such as pausing execution or inspecting intermediate task states [16].

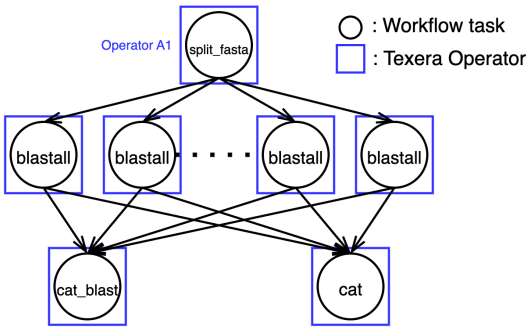

Figure 6: *Encapsulating each task as a Texera operator*

The second approach involves encapsulating each workflow task as a separate Texera operator. As depicted in Figure 6, each blue box corresponds to an individual Texera operator encapsulating a single workflow task. For instance, operator A1, illustrated by the rectangle containing the task labeled *split_fasta*, represents a singular operation. Using this strategy, each task, such as *blastall* and *cat_blast*, is independently managed by its own operator. This design offers semantic alignment with Texera's principle. It also enhances Texera's interactive workflow execution capabilities, such as detailed monitoring, pausing, and debugging at the granularity of individual tasks. Moreover, this approach provides improved observability for performance diagnostics. However, it introduces

increased complexity in managing workflows due to the potentially large number of operators, along with possible overhead associated with handling numerous individual operator instances.

For these experiments, we chose the second design, i.e., encapsulating each workflow task as an individual Texera operator, as it aligns closely with our experimental objectives of effectively showcasing Texera's performance capabilities and its unique strength in providing detailed, task-level granularity for monitoring. This design decision introduced additional challenges:

Texera Interfaces. Texera provides two interfaces for workflow execution: a user-friendly drag-and-drop GUI interface and a command-line based Python client. While the GUI interface simplifies workflow creation and interaction, workflows generated by WfBench, which can consist of hundreds or thousands of operators, may be cumbersome to manage visually due to the scale and complexity involved. To address this challenge, we leverage its Python client, which enables users to submit workflows defined in a JSON format directly to the engine via the command line, facilitating execution and retrieval of results without the overhead associated with a GUI. This dual-interface design ensures Texera remains accessible to both ordinary users, preferring GUI-based interaction, and advanced users who handle complex workflows using a command-line approach.

Region Scheduler Constraints. Texera executes workflows by segmenting them into regions [18], defined as weakly connected subgraphs in which all operators execute concurrently in a pipelined manner without intermediate materialization. Regions are separated by edges marked as materialized (typically corresponding to blocking operations). In our configuration for WfBench workflows, we treated each task as an individual region. Although forming each task into a separate region introduces additional scheduling overhead, this overhead occurs during the compilation and scheduling phase, without significantly affecting the execution phase. Texera's default region scheduler operates sequentially, executing regions one at a time. This scheduling strategy impeded the intended parallel execution characteristics of WfBench-generated workflows. To address this limitation, we improved the region scheduler to support parallel execution. Regions without direct dependencies could execute concurrently, significantly enhancing execution parallelism and overall workflow efficiency.

Our testing revealed another critical bottleneck: concurrently executing a large number of regions imposed excessive computational and resource burdens, causing system instability and performance degradation. To mitigate this issue, we introduced a configurable parameter, *max-concurrent-region*, allowing dynamic control over the maximum number of concurrently executing regions based on available computational resources. For instance, on a machine equipped with 12 CPU cores, we set *max-concurrent-region* to 12, aligning parallel region execution directly with hardware capacity. Additionally, we optimized resource utilization by immediately releasing resources upon task completion, significantly reducing computational strain. In future work, we plan to implement automatic detection of hardware capabilities, enabling dynamic configuration without manual intervention.

Optimization of UDF Operators. Initially, our workflows employed Python user-defined-function (UDF) operators within Texera. Each Python UDF operator instantiated its own separate Python Virtual Machine (PVM). For workflows consisting of thousands of tasks, this resulted in substantial overhead due to the large number of independent PVMs being created and managed. To address this issue, we changed the implementation from Python UDF operators to Java UDF operators. Unlike their Python counterparts, Java UDF operators run within a single shared Java Virtual Machine (JVM), significantly reducing resource consumption associated with virtual machine instantiation. Building further upon this improvement, we explored deeper integration using Java native operators, which embed tasks directly within Texera's native execution framework. This integration eliminates additional function invocation overhead inherent in UDF operators, providing further performance gains.

5 Experiments

In this section we report the experimental results.

5.1 Settings

Our experimental evaluation was designed to rigorously demonstrate Texera's performance across diverse computational profiles using WfBench workflows, specifically focusing on BLAST and Epigenomics tasks. To systematically analyze Texera's adaptability and efficiency, we defined three distinct resource configuration scenarios: CPU-intensive, balanced, and data-intensive. Each scenario leveraged specific parameters provided by WfBench, enabling precise control over task characteristics and ensuring meaningful, reproducible results. The primary parameters selected for this evaluation were *cpu_work* and *data*, chosen to highlight Texera's computational and data-handling capabilities. These parameters simulate realistic workloads common in scientific computing, allowing Texera's strengths and potential bottlenecks to be thoroughly assessed. The following are the parameters' description and rationale.

cpu_work: This parameter determines the computational complexity of each workflow task, controlled through WfBench's built-in CPU benchmarking executable [6], which incrementally computes a more precise value of π. Selecting varying levels of *cpu_work* allowed us to clearly observe how Texera scaled computationally and managed CPU resources under different intensity levels. Higher values of *cpu_work* resulted in greater computational demand, providing insights into Texera's capabilities in handling compute-intensive scientific tasks.

data: This parameter defines the total volume (MB) of data processed by the workflow. It is evenly distributed across tasks within the workflow, effectively simulating realistic I/O workloads. By varying *data*, we explicitly evaluated Texera's ability to manage I/O-intensive tasks, testing its capability to handle memory and disk operations, which is crucial for scientific workflows involving extensive data manipulations.

To systematically cover the spectrum of real-world computational demands, we selected the following scenarios based on a combination of these parameters:

- CPU-intensive (cpu_work = 600, data = 10): It was chosen to stress test Texera's computational performance, where tasks were heavily compute-bound with minimal data interaction.
- Balanced (cpu_work = 200, data = 200): It provided an equilibrium scenario representative of typical scientific workflows where workloads were neither dominated by computation

nor by data transfer. This scenario tested Texera's balanced resource management, ensuring reliable performance under moderate real-world conditions.
- Data-intensive (cpu_work = 10, data = 600): This setting was selected to challenge Texera's data management and I/O efficiency, as tasks were dominated by substantial data handling, reflecting workflows common in data-rich scientific applications such as genomic sequencing and bioinformatics.

Using this setting, we ensured the comprehensiveness of the evaluation. The contrasts and the balanced scenario in parameter values ensured broad coverage across typical and edge-case scenarios. For the experiments, we configured the BLAST and Epigenomics workflows across three parameter modes—CPU-intensive, balanced, and data-intensive—and three task sizes (45, 105, and 305 tasks), resulting in a total of 18 combinations to thoroughly test Texera. The experiments were conducted using two hardware platforms: (1) *MacBook Pro (macOS 15.0.1, arm64 architecture):* which was equipped with an Apple M3 Pro chip featuring 12 cores (6 performance cores and 6 efficiency cores) and 36 GB of memory. This setup represented modern consumer-grade hardware. (2) *Dell PowerEdge T330 Server (Ubuntu 22.04.4 LTS, x86_64 architecture):* which was configured with an Intel Xeon E3-1240 v6 quad-core processor (with hyper-threading, totaling 8 logical cores), 46.88 GB of RAM, and substantial storage capabilities. This setup represented a typical computing server environment. Choosing these two distinct environments allowed us to assess Texera's adaptability and robustness across both consumer-oriented systems and professional computing infrastructures. With these two platforms, the total number of experimental combinations increased to 36, each executed for five iterations to ensure statistically rigorous and reliable evaluations.

As mentioned earlier, the *max-concurrent-region* parameter in Texera controls the maximum number of workflow regions [18] executing simultaneously, effectively managing computational resource allocation and ensuring efficient execution. We configured *max-concurrent-region* based on the available computational resources of each platform. Specifically, we set it to 12 for the MacBook Pro to align with its 12 CPU cores and set it to 4 for the Dell server, considering its 4 physical cores.

Figure 7 illustrates the effect of *max-concurrent-region* clearly using the 45-task balanced mode of BLAST and Epigenomics workflows executed on macOS. In this case, each workflow task represented a single region. Regions within the same layer were executed concurrently up to the configured limit (12 in this case). If the number of regions in a single layer exceeded the configured limit, a maximum of 12 regions would execute simultaneously, demonstrating efficient parallel execution aligned with hardware resources. Additionally, WfBench managed the life-cycle of its benchmark processes by terminating them upon completion of each run, ensuring consistent and clean experimental conditions. The source code and experimental setup are available at [23].

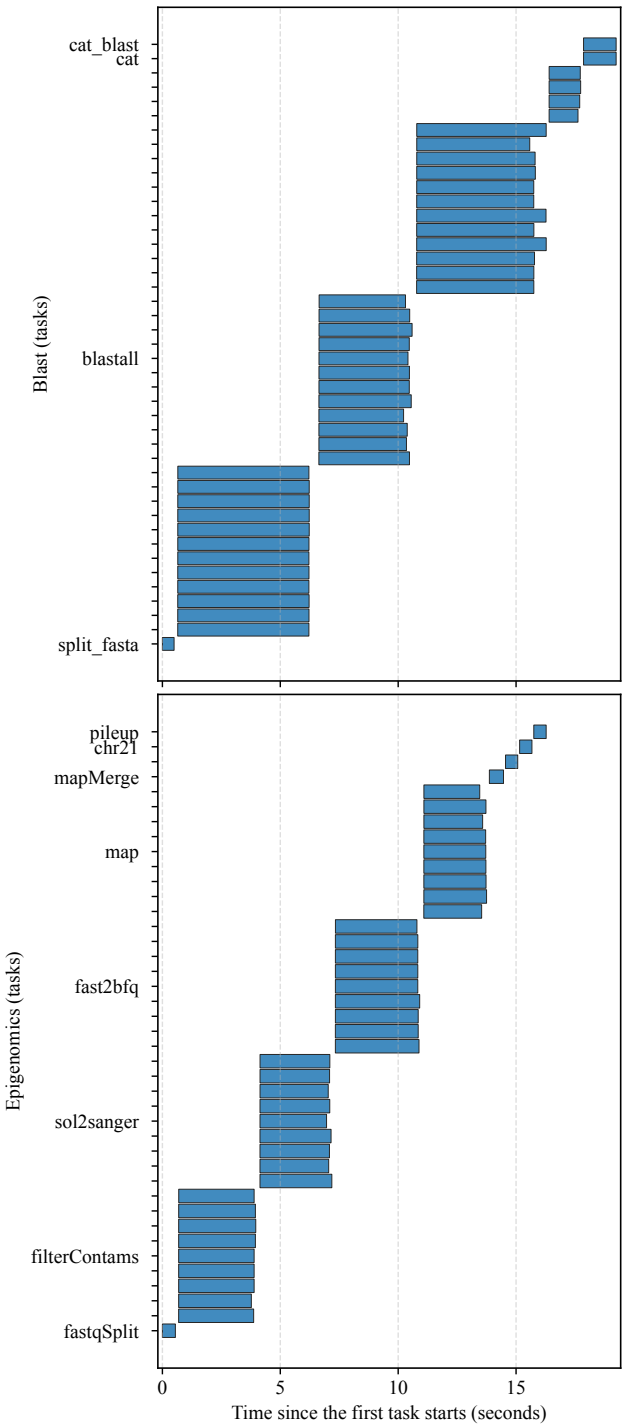

Figure 7: Concurrent region execution for BLAST and Epigenomics workflows (45-task balanced mode) on macOS demonstrating parallelization for independent workflow tasks

5.2 Execution Time

Figure 8 shows the execution times for the two workflows across three computational combinations (data-intensive, balance, CPU-intensive) and task sizes (45, 105, 305 tasks) on the two platforms. From the results, we had the following observations.

(1) *Impact of Computational Intensity*: The results consistently showed that CPU-intensive scenarios had significantly longer execution times compared to the other two scenarios. For example, in the 305-task configuration, the BLAST workflow on the Ubuntu

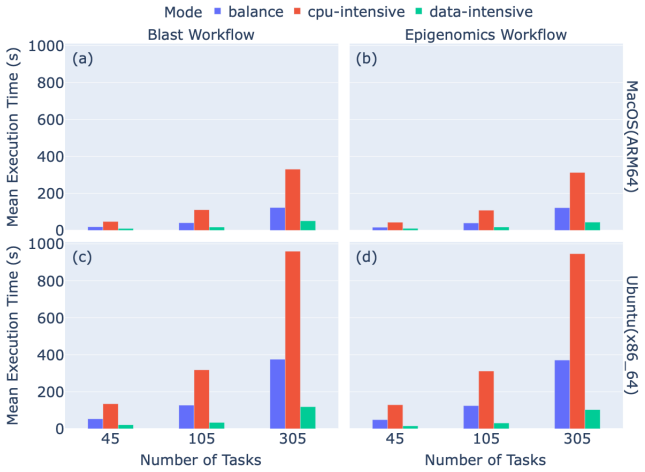

Figure 8: Execution times of BLAST and Epigenomics workflows under balanced, CPU-intensive, and data-intensive setups with task sizes 45, 105, and 305 on macOS (ARM64) and Ubuntu (x86_64).

platform (panel c) showed an average execution time of 960.05 seconds in the CPU-intensive mode, whereas it completed in 375.70 and 118.82 seconds in the balanced mode and data-intensive mode, respectively. Similarly, on macOS (panel a), the execution time of the 305-task BLAST workflow was 330.83 seconds for the CPU-intensive mode, compared to 123.20 seconds in the balanced mode and 51.58 seconds in the data-intensive mode.

(2) *Workflow Comparison*: Both workflows exhibited similar execution time patterns across all the experimental scenarios, demonstrating Texera's robustness and consistency in handling diverse workflow types. These similarities likely arose from their reliance on parallel task execution. Additionally, when the task size for both workflows increased to 350, their execution times scaled as expected, demonstrating Texera's reliable scalability.

(3) *Platform Comparison*: Comparing the two hardware platforms revealed similar performance patterns but distinct execution times. Tasks executed on Ubuntu consistently took longer compared to macOS, primarily due to hardware differences, most notably, the number of CPU cores. On the Ubuntu server (panels c and d), equipped with fewer physical cores (4 physical cores), the workflows had longer execution times compared to the macOS platform (panels a and b) with 12 cores. These results demonstrated Texera's capability to run workflows across different hardware settings, resource configurations, and operating systems.

Overall, the results showed that Texera can effectively manage computational and data-intensive workloads in various task configurations and platform environments.

5.3 CPU Usage Pattern

Figure 9 illustrates the CPU usage patterns during the execution of BLAST and Epigenomics workflows on the macOS platform, as captured by the "Activity Monitor" tool. Each sub-figure represented a 60-second execution segment for a specific combination. The usage was categorized into "User CPU" (blue), representing processing time consumed by user-level applications and computations, and

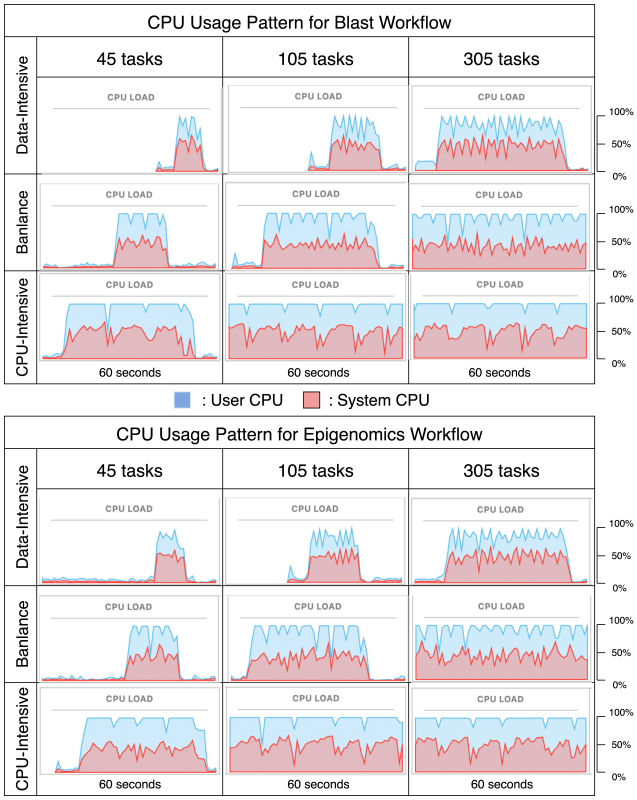

Figure 9: CPU usage patterns for BLAST and Epigenomics workflows executed under data-intensive, balanced, and CPU-intensive modes with varying task sizes (45, 105, 305 tasks) on the macOS platform.

"System CPU" (red), indicating kernel-level operations such as task scheduling, I/O management, and other system-managed processes. The Y-axis represented aggregate utilization across all the available cores (12 cores for the macOS platform), where 100% indicated full utilization of all cores combined.

Notably, the patterns across different task sizes (45, 105, and 305) were consistent within each computational mode, confirming Texera's reliable resource utilization. Moreover, there was consistency in CPU usage patterns across both workflows when they were executed under identical computational modes despite their different internal structures. This consistency was primarily attributed to the identical configuration of the *max-concurrent-region* parameter, limiting concurrent executions of regions to 12 in this case.

An important observation was the periodic gaps in CPU utilization, seen as brief drops in usage patterns. These gaps corresponded to transitions between batches of concurrent regions. Specifically, after a batch of regions completed their execution, the system needed to initiate the next batch. Furthermore, the duration of continuous high CPU utilization clearly varied according to the computational intensity defined by the *cpu_work* parameter. CPU-intensive tasks sustained prolonged periods of peak CPU usage, reflecting extensive computational demands. In contrast, data-intensive and balanced tasks demonstrated shorter and more intermittent peak utilization, correlating with their computational

characteristics. These results demonstrated Texera's capability to effectively manage computational resources according to varying workload intensities, ensuring stable and predictable performance across diverse workflow configurations.

6 Conclusions and Future Work

In this work, we demonstrated the integration of Texera with the WfBench benchmarking framework, highlighting Texera's capability to handle diverse workflow patterns effectively. Our evaluation using two representative workflows, BLAST and Epigenomics, across diverse profiles, task sizes, and hardware platforms confirmed Texera's adaptability, resource efficiency, and stable performance. Future work includes expanding the evaluation framework to directly compare Texera with other workflow systems using identical WfBench workflows, offering comprehensive comparative insights. Additionally, we intend to further investigate the relationship between the *max-concurrent-region* parameter and the overall workflow performance, aiming to dynamically optimize resource scheduling and execution efficiency.

Acknowledgments

The work of Wang and Li is funded by the National Science Foundation award III-2107150 and the National Institutes of Health award 1U01AG076791-01. The work of Maheshwari is funded by ORNL which is supported by DOE's Office of Science under Contract No. DE-AC05-00OR22725.

References

[1] Stephen F Altschul, Warren Gish, Webb Miller, Eugene W Myers, and David J Lipman. 1990. Basic local alignment search tool. *Journal of molecular biology* 215, 3 (1990), 403–410.

[2] Peter Amstutz, Maxim Mikheev, Michael R. Crusoe, Nebojša Tijanić, Samuel Lampa, et al. 2024. Existing Workflow Systems. https://s.apache.org/existing-workflow-systems. Common Workflow Language wiki, GitHub. Updated 2024-08-18, accessed 2024-08-18.

[3] Apache Airflow Community. 2025. Apache Airflow. https://airflow.apache.org/. Accessed 2025-06-29.

[4] Yadu N. Babuji, Kyle Chard, Ian T. Foster, Daniel S. Katz, Mike Wilde, Anna Woodard, and Justin M. Wozniak. 2018. Parsl: Scalable Parallel Scripting in Python. In *Proceedings of the 10th International Workshop on Science Gateways, Edinburgh, Scotland, UK, 13-15 June, 2018 (CEUR Workshop Proceedings, Vol. 2357)*, Malcolm P. Atkinson and Sandra Gesing (Eds.). CEUR-WS.org. https://ceur-ws.org/Vol-2357/paper11.pdf

[5] Henri Casanova, Kyle Berney, Serge Chastel, and Rafael Ferreira Da Silva. 2023. WfCommons: Data Collection and Runtime Experiments using Multiple Workflow Systems. In *2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 1870–1875.

[6] Tainã Coleman, Henri Casanova, Ketan Maheshwari, Loïc Pottier, Sean R. Wilkinson, Justin M. Wozniak, Frédéric Suter, Mallikarjun Shankar, and Rafael Ferreira da Silva. 2022. WfBench: Automated Generation of Scientific Workflow Benchmarks. In *IEEE/ACM International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems, PMBS@SC 2022, Dallas, TX, USA, November 13-18, 2022*. IEEE, 100–111. doi:10.1109/PMBS56514.2022.00014

[7] Tainã Coleman, Henri Casanova, Loïc Pottier, Manav Kaushik, Ewa Deelman, and Rafael Ferreira da Silva. 2022. WfCommons: A framework for enabling scientific workflow research and development. *Future generation computer systems* 128 (2022), 16–27.

[8] The Galaxy Community. 2024. The Galaxy platform for accessible, reproducible, and collaborative data analyses: 2024 update. *Nucleic Acids Research* 52, W1 (05 2024), W83–W94. arXiv:https://academic.oup.com/nar/article-pdf/52/W1/W83/58436305/gkae410.pdf doi:10.1093/nar/gkae410

[9] Ewa Deelman, Karan Vahi, Gideon Juve, Mats Rynge, Scott Callaghan, Philip Maechling, Rajiv Mayani, Weiwei Chen, Rafael Ferreira da Silva, Miron Livny, and R. Kent Wenger. 2015. Pegasus, a workflow management system for science automation. *Future Gener. Comput. Syst.* 46 (2015), 17–35. doi:10.1016/J.FUTURE.2014.10.008

[10] Philipp Diercks, Dennis Gläser, Ontje Lünsdorf, Michael Selzer, Bernd Flemisch, and Jörg F. Unger. 2022. Evaluation of tools for describing, reproducing and reusing scientific workflows. *CoRR* abs/2211.06429 (2022). arXiv:2211.06429 doi:10.48550/ARXIV.2211.06429

[11] Yunyan Ding, Yicong Huang, Pan Gao, Andy Thai, Atchuth Naveen Chilaparasetti, M Gopi, Xiangmin Xu, and Chen Li. 2024. Brain image data processing using collaborative data workflows on Texera. *Frontiers in Neural Circuits* 18 (2024), 1398884.

[12] Lu He, Changyang He, Tera L Reynolds, Qiushi Bai, Yicong Huang, Chen Li, Kai Zheng, and Yunan Chen. 2021. Why do people oppose mask wearing? A comprehensive analysis of US tweets during the COVID-19 pandemic. *Journal of the American Medical Informatics Association* 28, 7 (2021), 1564–1573.

[13] Workflows Community Initiative. 2025. Workflows Community initiative. https://workflows.community/

[14] Keith R. Jackson, Lavanya Ramakrishnan, Krishna Muriki, Shane Canon, Shreyas Cholia, John Shalf, Harvey J. Wasserman, and Nicholas J. Wright. 2010. Performance Analysis of High Performance Computing Applications on the Amazon Web Services Cloud. In *Cloud Computing, Second International Conference, CloudCom 2010, November 30 - December 3, 2010, Indianapolis, Indiana, USA, Proceedings*. IEEE Computer Society, 159–168. doi:10.1109/CLOUDCOM.2010.69

[15] Jessie WY Ko, Shengquan Ni, Alexander Taylor, Xiusi Chen, Yicong Huang, Avinash Kumar, Sadeem Alsudais, Zuozhi Wang, Xiaozhen Liu, Wei Wang, et al. 2024. How the experience of California wildfires shape Twitter climate change framings. *Climatic Change* 177, 1 (2024), 17.

[16] Avinash Kumar, Zuozhi Wang, Shengquan Ni, and Chen Li. 2020. Amber: A Debuggable Dataflow System Based on the Actor Model. *Proc. VLDB Endow.* 13, 5 (2020), 740–753. doi:10.14778/3377369.3377381

[17] Jeremy Leipzig. 2017. A review of bioinformatic pipeline frameworks. *Briefings Bioinform.* 18, 3 (2017), 530–536. doi:10.1093/BIB/BBW020

[18] Xiaozhen Liu, Yicong Huang, Xinyuan Lin, Avinash Kumar, Sadeem Alsudais, and Chen Li. 2024. Pasta: A Cost-Based Optimizer for Generating Pipelining Schedules for Dataflow DAGs. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–26.

[19] Xiaozhen Liu, Zuozhi Wang, Shengquan Ni, Sadeem Alsudais, Yicong Huang, Avinash Kumar, and Chen Li. 2022. Demonstration of Collaborative and Interactive Workflow-Based Data Analytics in Texera. *Proc. VLDB Endow.* 15, 12 (2022), 3738–3741. https://www.vldb.org/pvldb/vol15/p3738-liu.pdf

[20] Matthew Rocklin. 2015. Dask: Parallel Computation with Blocked algorithms and Task Scheduling. In *Proceedings of the 14th Python in Science Conference 2015 (SciPy 2015), Austin, Texas, July 6 - 12, 2015*, Kathryn Huff and James Bergstra (Eds.). scipy.org, 126–132. doi:10.25080/MAJORA-7B98E3ED-013

[21] Texera Project. 2025. Texera. https://github.com/apache/texera Accessed 2025-08-10.

[22] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. 2017. Nextflow enables reproducible computational workflows. *Nature Biotechnology* 35, 4 (2017), 316–319. doi:10.1038/nbt.3820

[23] Meng Wang. 2025. wisdom2025-WFBENCH-and-Texera-integration. https://github.com/Texera/WISDOM2025-WfBench-and-Texera-Integration

[24] Zuozhi Wang, Yicong Huang, Shengquan Ni, Avinash Kumar, Sadeem Alsudais, Xiaozhen Liu, Xinyuan Lin, Yunyan Ding, and Chen Li. 2024. Texera: A System for Collaborative and Interactive Data Analytics Using Workflows. *Proc. VLDB Endow.* 17, 11 (2024), 3580–3588. doi:10.14778/3681954.3682022

[25] WfCommons Project. 2025. *The WfCommons Project — WfCommons 1.2-dev Documentation: Introduction.* https://docs.wfcommons.org/en/latest/introduction.html Accessed 2025-06-30.

[26] Katherine Wolstencroft, Robert Haines, Donal Fellows, Alan R. Williams, David Withers, Stuart Owen, Stian Soiland-Reyes, Ian Dunlop, Aleksandra Nenadic, Paul Fisher, Jiten Bhagat, Khalid Belhajjame, Finn Bacall, Alex R. Hardisty, Abraham Nieva de la Hidalga, Maria P. Balcazar Vargas, Shoaib Sufi, and Carole A. Goble. 2013. The Taverna workflow suite: designing and executing workflows of Web Services on the desktop, web or in the cloud. *Nucleic Acids Res.* 41, Webserver-Issue (2013), 557–561. doi:10.1093/NAR/GKT328

[27] Justin M. Wozniak, Timothy G. Armstrong, Michael Wilde, Daniel S. Katz, Ewing L. Lusk, and Ian T. Foster. 2013. Swift/T: Large-Scale Application Composition via Distributed-Memory Dataflow Processing. In *13th IEEE/ACM International Symposium on Cluster, Cloud, and Grid Computing, CCGrid 2013, Delft, Netherlands, May 13-16, 2013*. IEEE Computer Society, 95–102. doi:10.1109/CCGRID.2013.99